VOL. 22

C#

Types, records, LINQ, async and the web, written for .NET 8

Source	Microsoft Learn (fallback for dotnet.microsoft.com/learn/csharp)
Generated	2026-10-10
Level	Intermediate
Estimated study time	30 hours
Language	English
Volume	Vol. 22 - C#

This volume is personal study material. The notes, examples, exercises and tests were written from scratch while following the source site's learning path. No content is affiliated with, endorsed by, or copied from the source site or the technology's owner.

Table of contents

How to use this volume 4

Chapter map 4

Chapter 1 - Hello C#: setup, the toolchain and your first program 6

1.1 Summary 6

1.2 Concepts 6

What C# is 6

The pipeline 6

The SDK and the command line 7

Project layout 7

Top-level statements 8

Reading and writing the console 8

1.3 Examples 8

1.4 Common mistakes and best practices 10

1.5 Practice exercises 11

Solutions 11

1.6 Mini test 11

Answer key and explanations 12

Chapter 2 - Types, variables and operators 12

2.1 Summary 12

2.2 Concepts 13

The built-in types 13

Value types and reference types 13

Declaring variables 14

Converting between types 14

Operator families 14

Precedence 15

2.3 Examples 15

2.4 Common mistakes and best practices 18

2.5 Practice exercises 18

Solutions 19

2.6 Mini test 19

Answer key and explanations 19

Chapter 3 - Control flow: decisions and loops 20

3.1 Summary . . . 20
3.2 Concepts . . . 20
How `if` decides . . . 20
Boolean building blocks . . . 21
The ternary operator . . . 21
The `switch` statement . . . 22
The `while` and `do` loops . . . 22
The `for` loop . . . 23
The `foreach` loop . . . 23
Jump statements and scope . . . 23
3.3 Examples . . . 24
3.4 Common mistakes and best practices . . . 27
3.5 Practice exercises . . . 28
Solutions . . . 28
3.6 Mini test . . . 28
Answer key and explanations . . . 29

How to use this volume

Prerequisites. No C# experience is assumed. You should be comfortable with the idea of a variable, a loop and a function in any language, because this volume explains C# itself rather than programming in general. Knowing a statically typed language such as Java, TypeScript, Rust or Go makes several chapters faster to read; knowing only a dynamic language such as Python or JavaScript is enough, since every type and every keyword is introduced here.

Estimated study time. About 30 hours: roughly two hours per chapter plus the time you spend on your own experiments. Chapters 1 to 8 build the core of the language, chapters 9 to 12 cover the modern idioms you meet in real code bases, and chapters 13 and 14 connect the language to files, JSON, databases and the web.

How the chapters work. Every chapter opens with a summary, explains the concepts, shows numbered examples with the output they produce, lists the common mistakes with the best practice that fixes each one, and ends with practice exercises, their solutions, and a mini test with an answer key.

How to run the code. Every example is a complete console program. Create a project once and replace `Program.cs` with the code you want to try:

shell

```shell
dotnet new console -o play -n play
cd play
dotnet run
```

output

```text
Hello, C#!
```

Blocks tagged `cs` are C# source, blocks tagged `text` show the expected output, blocks tagged `xml` are project or configuration files, and blocks tagged `bash` are terminal commands. All examples target C# 12 on .NET 8, which means they use top-level statements, collection expressions such as `[1, 2, 3]`, records and pattern matching. The output blocks were not captured on the machine where this volume was written, because no .NET SDK was installed there, so they are the values the documented behaviour of each API produces; run the programs yourself to confirm them. Number formatting assumes an English culture, where the decimal separator is a full stop.

About the source. The syllabus follows the free "Get started with C#" learning paths and the official C# guide published by Microsoft Learn. Only the topic names were taken from that material; every sentence, program, exercise and test in this volume was written from scratch.

Chapter map

#	Chapter	You will learn
1	Hello C#: setup, the toolchain and your first program	how C# compiles and runs, `dotnet` commands, the project file, top-level statements, console input and output

#	Chapter	You will learn
2	Types, variables and operators	value and reference types, the built-in type table, `var` and `const`, conversions, casting, parsing, every operator group, precedence
3	Control flow: decisions and loops	`if` / `else if` / `else`, `switch` statements, ternary, `while`, `do`, `for`, `foreach`, `break`, `continue`, scope
4	Strings and text formatting	string immutability, interpolation, alignment and number formats, `StringBuilder`, splitting and joining, parsing, character helpers
5	Arrays, lists and collections	arrays, collection expressions, `List<T>`, `Dictionary<K,V>`, `HashSet<T>`, `Stack<T>`, `Queue<T>`, indexers, ranges
6	Methods and parameters	declaring methods, return values, `ref`, `out`, `in`, `params`, optional and named arguments, overloading, local functions, recursion
7	Object-oriented C#	classes, properties, constructors, records, inheritance, `virtual` and `override`, abstract classes, interfaces, enums, structs
8	Generics and nullable reference types	generic classes and methods, constraints, `string?` and `int?`, null-state analysis, `??`, `?.`, the null-forgiving operator
9	Pattern matching and switch expressions	type, constant, relational, logical, property, positional and list patterns, `switch` expressions, guards
10	Exceptions and error handling	exception hierarchy, `try` / `catch` / `finally`, exception filters, custom exceptions, `using` and `IDisposable`
11	LINQ	lambdas, `Where`, `Select`, `OrderBy`, `GroupBy`, `Join`, aggregates, query syntax, deferred execution, `Func` and `Action`
12	Async/await and Task	`Task` and `Task<T>`, `async Main`, `await`, `Task.WhenAll`, async exceptions, async streams, cancellation
13	Files, JSON and System.IO	`Path`, `Directory`, `File`, readers and writers, `System.Text.Json` serialization and deserialization
14	EF Core, minimal APIs, tooling and testing	`DbContext` and migrations, ASP.NET Core minimal APIs, the `dotnet` CLI, debugging, unit tests with xUnit
-	Cheat sheet	compact reference tables for the whole volume
-	Final test	25 questions with answers and explanations

Chapter 1 - Hello C#: setup, the toolchain and your first program

1.1 Summary

- C# is a statically typed, object-oriented language whose code runs on the .NET runtime.
- Source files are compiled to intermediate language, which the just-in-time compiler turns into machine code when the program starts.
- The .NET SDK bundles the compiler, the runtime and the `dotnet` command line tool.
- `dotnet new console` creates a project, `dotnet build` compiles it, and `dotnet run` builds then starts it.
- Modern programs use top-level statements, so the compiler generates the enclosing class and the entry point for you.
- The project file `.csproj` records the target framework together with features such as nullable and implicit usings.
- `Console.WriteLine` writes a line, `Console.Write` writes without a line break, and `Console.ReadLine` reads one line of input.
- Naming conventions keep code readable: PascalCase for types and methods, camelCase for locals and parameters.
- Compilation errors produce a code such as CS1002 with a file name and a line number, so they point straight at the defect.
- Every .NET type derives from `System.Object`, which is why every value understands `ToString`.

1.2 Concepts

What C# is

C# is a general-purpose language with static typing: every variable, constant, parameter and expression has a type that the compiler checks before the program runs. Type safety turns a large class of runtime crashes into build errors. The language is managed, which means memory is reclaimed by a garbage collector, and object-oriented, which means you model problems with classes, records, interfaces and inheritance.

Stage	Tool	Result
Compile time	Roslyn compiler	an assembly with IL and metadata
Start time	just-in-time compiler	machine code for the methods that run
Run time	.NET runtime	memory management, exceptions, threads

The pipeline

output

```
 Program.cs           Program.dll            this process
+-----------+   C#   +----------------+      +--------------+
| statements| -----> | IL + metadata  | ---> | native code  |
+-----------+        +----------------+ JIT  +--------------+
```

The compiler does not produce native code for a normal build: it produces intermediate language that any .NET runtime can execute, so one assembly runs on Windows, Linux and macOS. The just-in-time compiler translates a method the first time it is called, which is why the first call of a method is slightly slower than later calls. Ahead-of-time publishing exists for start-up-sensitive programs, but ordinary development follows the picture above.

The SDK and the command line

Command	What it does
`dotnet --version`	prints the installed SDK version
`dotnet new console -o play`	creates a console project in the folder `play`
`dotnet build`	compiles into `bin/Debug/<framework>/`
`dotnet run`	builds if needed, then runs the program
`dotnet watch`	rebuilds and restarts when a file changes
`dotnet test`	builds and runs unit tests
`dotnet format`	applies the project code-style rules
`dotnet publish -c Release`	produces a folder ready to deploy
`dotnet add package Name`	adds a NuGet package reference

A console project needs no network access beyond the first restore, and the SDK restores the base class libraries from the local cache.

Project layout

output

```
play/
  play.csproj        project file: framework and features
  Program.cs         top-level statements live here
  obj/               generated build inputs
  bin/
    Debug/
      net8.0/
        play.dll     the compiled assembly
```

The project file is short, and the template writes it for you:

XML

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net8.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>

</Project>
```

Three settings matter for this volume. `TargetFramework` selects the version of the base library and the language version. `ImplicitUsings` adds a set of `using` directives, including `System`, `System.Collections.Generic`, `System.Linq` and `System.IO`, so the examples do not repeat them. `Nullable` turns on the compile-time null analysis used from chapter 8 onwards. Web projects use `Microsoft.NET.Sdk.Web`, which adds the ASP.NET Core framework reference described in chapter 14.

Top-level statements

A program is a sequence of statements. Since C# 9 the compiler accepts them at the top of the file and wraps them in a generated class and `Main` method, which keeps small programs free of ceremony.

C#

```
Console.WriteLine("first statement");
Console.WriteLine("second statement");
```

Two rules keep this legal. Only one file in the project may contain top-level statements, and that file must put `using` directives first, then any type or namespace declarations, and then the statements. Local functions are allowed inside top-level statements, which is how the examples in this volume stay in one file.

Reading and writing the console

Member	Effect
`Console.WriteLine("text")`	writes the text and a line break
`Console.Write("text")`	writes the text, the cursor stays on the line
`Console.ReadLine()`	reads one line, returns `string?`
`Console.Error.WriteLine("x")`	writes to the error stream

`Console.ReadLine` returns `string?` because the user may close the input, and the result is always text, so numbers must be converted before arithmetic. Chapter 2 covers the conversion methods.

1.3 Examples

Example 1: the smallest complete program.

C#

```
1  Console.WriteLine("Hello, C#!");
```

output

```
1  Hello, C#!
```

The compiler adds the enclosing class and the entry point, so the file contains one statement and nothing else.

Example 2: values and interpolation.

C#

```
1  string name = "Ada";
2  int year = 2026;
3  Console.WriteLine($"{name} started learning C# in {year}.");
```

output

```
1  Ada started learning C# in 2026.
```

The `$` before the string turns it into an interpolated string, and each pair of braces holds an expression that is evaluated and formatted at that position.

Example 3: reading a line of input.

C#

```
1  Console.Write("Your name: ");
2  string? name = Console.ReadLine();
3  Console.WriteLine($"Hello, {name}!");
```

output

```
1  Your name: Hi
2  Hello, Hi!
```

The output assumes the user typed `Hi` and pressed Enter. `Console.Write` leaves the cursor on the same line, so the echoed input appears right after the prompt.

Example 4: top-level statements with a local function.

C#

```
1  int total = Sum(2, 3, 4);
2  Console.WriteLine($"total is {total}");
3
4  int Sum(params int[] values)
5  {
6      int running = 0;
7      foreach (int value in values)
8      {
9          running += value;
10     }
11     return running;
12 }
```

output

```
total is 9
```

A local function is declared inside the top-level code and can be called before the line that declares it, because the whole file is one method body.

Example 5: the same program with an explicit entry point.

C#

```csharp
using System;

namespace Demo;

internal class Program
{
    private static void Main(string[] args)
    {
        Console.WriteLine("from Main");
        Console.WriteLine($"arguments: {args.Length}");
    }
}
```

output

```
from Main
arguments: 0
```

`Main` may return `void`, `int` or `Task`, and `args` carries the command line arguments. Top-level statements compile to exactly this shape, generated for you.

Example 6: the commands that build and run the project.

shell

```shell
dotnet new console -o play -n play
cd play
dotnet run
```

output

```
The template "Console App" was created successfully.

Processing post-creation actions...
  Restore succeeded.
Hello, C#!
```

The welcome text and the exact wording of the template message depend on the installed SDK; the last line is the output of `Program.cs`.

1.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Forgetting the semicolon	the build stops with CS1002 and nothing runs	end every statement with `;` and let the editor format it
Storing `1.5` in an `int`	the literal and the type disagree, so the compiler rejects it	choose the type that matches the value, or write `1`

Mistake	Why it hurts	Best practice
Doing arithmetic on `Console.ReadLine()`	the result is text, and `+` concatenates instead of adding	convert with `int.Parse` or `int.TryParse` first
Two files with top-level statements	the build reports a duplicate entry point, CS8802	keep one entry point and move the rest into classes
Naming the project file differently from the folder	confusing paths in build output and scripts	accept the default name the template proposes
Writing an unescaped quote inside a string	the string ends early and the rest becomes syntax noise	use `\"` or a verbatim string
Ignoring culture when formatting numbers	`3.5` prints as `3,5` on a Spanish or German machine	format with `CultureInfo.InvariantCulture` for stable output

1.5 Practice exercises

1. Write a program that prints your name and the current year in one sentence using an interpolated string.
2. Which `dotnet` command builds the project, and which one builds and starts it?
3. Rewrite the explicit `Program` / `Main` form as top-level statements.
4. A program that should print `3.5` prints `3`. Name the two most likely causes.
5. Why is `int.TryParse` preferred over `int.Parse` for text a user types?

Solutions

1. `Console.WriteLine($"Ana is learning C# in {2026}.");` prints `Ana is learning C# in 2026.`
2. `dotnet build` compiles; `dotnet run` compiles when needed and then executes the program.
3. Remove the class, the `using` directive and `Main`, then leave the statements in the file: `Console.WriteLine("from Main");`
4. Either both operands are integers, so `/` performs integer division, or the result was assigned to an `int` variable. Divide a `double` by a `double`, or cast one operand with `(double)`.
5. `int.Parse` throws when the text is not a number, while `int.TryParse` returns `false` and leaves the variable at its default, so the program can report the problem and continue.

1.6 Mini test

1. What does the C# compiler produce for a normal build? A) Native machine code B) Intermediate language plus metadata C) JavaScript D) Byte code for a virtual machine written in Python
2. Which command compiles and starts a console program? A) `dotnet build` B) `dotnet run` C) `dotnet new console` D) `dotnet publish`
3. Which project setting enables the compile-time null analysis? A) `<Nullable>enable</Nullable>` B) `<ImplicitUsings>enable</ImplicitUsings>` C)

`<TargetFramework>net8.0</TargetFramework>` D) `<OutputType>Exe</OutputType>`

4. True or false: a project may contain two files with top-level statements.
5. What is the return type of `Console.ReadLine()`? A) `string` B) `string?` C) `int` D) `char`
6. What does `$"{2 + 3}"` evaluate to? A) `5` B) `23` C) `2 + 3` D) a compile error, because strings cannot hold expressions
7. Which naming convention fits a local variable? A) PascalCase B) camelCase C) SCREAMING_CASE D) kebab-case

Answer key and explanations

1. B - normal builds emit IL in an assembly; the just-in-time compiler produces machine code when the program runs.
2. B - `dotnet run` builds when needed and then executes the result.
3. A - the `Nullable` property turns on nullable reference type analysis.
4. False - only one file may hold the entry point, otherwise the build reports a duplicate entry point error.
5. B - reading may fail, so the type is nullable and must be checked before use.
6. A - braces hold an expression, so the addition happens before formatting.
7. B - locals and parameters use camelCase, while types and methods use PascalCase.

Chapter 2 - Types, variables and operators

2.1 Summary

- Every variable, constant and expression has a type that the compiler verifies before the program runs.
- Value types hold their data directly, while reference types hold a pointer to an object on the managed heap.
- C# has a fixed set of built-in types, each with a keyword, a size and a range.
- `var` asks the compiler to infer the type from the assigned value, and `const` marks a value that never changes.
- Implicit conversions keep data, explicit conversions with a cast may lose it, and `TryParse` turns text into numbers safely.
- Integer division truncates, so `7 / 2` is `3` and `7 / 2.0` is `3.5`.
- Arithmetic overflow wraps silently by default unless the code is compiled with `checked`.
- Operators group into arithmetic, assignment, comparison, logical, bitwise, shift and conditional families.
- Precedence decides how `2 + 3 * 4` parses, and parentheses remove all doubt.
- A format specifier controls how a value appears on screen, and the culture controls the separators.

2.2 Concepts

The built-in types

Keyword	Category	Size	Range or notes
`bool`	value	1 byte	`true` or `false`
`char`	value	2 bytes	one UTF-16 code unit, such as `a` or a newline
`byte`	value	1 byte	0 to 255
`sbyte`	value	1 byte	-128 to 127
`short`	value	2 bytes	-32768 to 32767
`ushort`	value	2 bytes	0 to 65535
`int`	value	4 bytes	about -2.1 billion to 2.1 billion
`uint`	value	4 bytes	0 to about 4.3 billion
`long`	value	8 bytes	about -9.2 quintillion to 9.2 quintillion
`ulong`	value	8 bytes	0 to about 1.8 quintillion
`float`	value	4 bytes	about 7 significant digits, suffix `f`
`double`	value	8 bytes	about 15 to 16 significant digits
`decimal`	value	16 bytes	about 28 to 29 digits, suffix `m`
`string`	reference	-	immutable sequence of characters
`object`	reference	-	the base of every type
`int[]`	reference	-	an array is a reference to a block of values

Choose `int` for whole numbers, `double` for scientific and general measurements, and `decimal` for money, where binary rounding would produce visible errors. Use `long` only when the range of `int` is genuinely too small.

Value types and reference types

A variable of a value type holds the data itself. Assigning it copies the data, so the two variables are independent afterwards. A variable of a reference type holds the address of an object on the heap, so two variables can point at the same object.

output

```
value type                reference type
+---------+               +---------+        +-------------------+
| total   | = 42          | list    | -----> | [0]=1 [1]=2 [2]=3  |
| copy    | = 42          | alias   | -----> |  (the same object)|
+---------+               +---------+        +-------------------+
```

Structs, enums and the numeric types are value types. Classes, arrays, strings, delegates and records are reference types.

Declaring variables

Form	Example	Meaning
explicit type	`int count = 3;`	the type is written in the source
inferred type	`var total = 3;`	the compiler uses the type of the initial value
constant	`const double Pi = 3.14159;`	fixed at compile time, cannot be reassigned
nullable value	`int? maybe = null;`	the value type plus the state "no value"
several at once	`int a = 1, b = 2;`	one type, several names

`var` is not a variant type: the variable keeps the type that the initial value had, and assigning an incompatible value is a compile error. A constant must be initialised with a compile-time constant expression.

Converting between types

Direction	How	Behaviour
widening	`int` to `long`	implicit, no data lost
narrowing	`double` to `int`	cast required, the fraction is discarded
numeric to text	`value.ToString()`	uses the current culture
text to number	`int.Parse(text)`	throws when the text is invalid
text to number, safe	`int.TryParse(text, out n)`	returns `false`, sets `n` to 0
arbitrary	`Convert.ToInt32(value)`	works through `IConvertible`

Casting never rounds: `(int)3.99` is `3`. Input that a user types should always use the `TryParse` form, because a failed conversion is a normal event and not an exceptional one.

Operator families

Family	Operators
arithmetic	`+ - * / %`
unary	`+ - ! ~ ++ --`
assignment	`= += -= *= /= %= &= ` ` = ^= <<= >>=` `
comparison	`== != < > <= >=`
logical	`&& ` ` & ^` `
shift	`<< >>`
conditional	`?: ?? ??=` and the null-conditional `?.`
type test	`is`, `as`, `typeof`

`&&` and `||` short-circuit: the right operand is evaluated only when it can change the result, so `text is not null && text.Length > 0` is safe even when `text` is null.

Precedence

From highest to lowest, the operators you use most often: primary and unary, then `* / %`, then `+ -`, then shifts, then the relational operators, then equality, then the bitwise operators, then `&&`, then `||`, then `??`, and finally the assignment operators. In practice, write parentheses whenever a reader would have to pause, because precedence solves the ambiguity only for the compiler.

C#

```
Console.WriteLine(2 + 3 * 4);                // 14
Console.WriteLine((2 + 3) * 4);              // 20
Console.WriteLine(10 - 4 - 3);               // 3, left to right
Console.WriteLine(10 / 5 * 2);               // 4, left to right
Console.WriteLine(true || false && false);   // True, && binds tighter
```

2.3 Examples

Example 1: declaring and inspecting types.

C#

```
int age = 30;
double height = 1.75;
decimal price = 9.99m;
bool active = true;
char grade = 'A';
string city = "Madrid";

var count = 5;
var names = new List<string>();

Console.WriteLine(count.GetType());   // System.Int32
Console.WriteLine(active.GetType());  // System.Boolean
Console.WriteLine(price.GetType());   // System.Decimal
Console.WriteLine(height.GetType());  // System.Double
```

output

```
System.Int32
System.Boolean
System.Decimal
System.Double
```

The runtime prints the full name of each type, which is the easiest way to check what `var` inferred.

Example 2: integer and floating division.

C#

```
int a = 7, b = 2;
Console.WriteLine(a / b);              // 3
Console.WriteLine((double)a / b);      // 3.5
Console.WriteLine(7.0 / 2);            // 3.5
Console.WriteLine(7 % 3);              // 1
Console.WriteLine(-7 % 3);             // -1
```

output

```
3
3.5
3.5
1
-1
```

`%` keeps the sign of the left operand, so it is a remainder rather than a modulo operator in the mathematical sense.

Example 3: overflow and `checked`.

C#

```
int largest = int.MaxValue;
Console.WriteLine(largest);            // 2147483647
Console.WriteLine(largest + 1);        // -2147483648
Console.WriteLine(checked(largest + 1));
```

output

```
2147483647
-2147483648
Unhandled exception. System.OverflowException: Arithmetic operation resulted in an overflow.
```

The default is unchecked arithmetic, for performance. Wrapping an expression or a block in `checked` turns silent overflow into an exception, which is what you want for money and counters. The last line is the message the runtime prints when the process is not hosted by a debugger.

Example 4: converting text to numbers.

C#

```
string text = "42";
Console.WriteLine(int.Parse(text) + 1);      // 43
Console.WriteLine(double.Parse("3.5"));      // 3.5

Console.WriteLine(int.TryParse("17", out int ok));    // True
Console.WriteLine(ok);                                // 17
Console.WriteLine(int.TryParse("abc", out int bad));  // False
Console.WriteLine(bad);                               // 0
```

output

```
43
3.5
True
17
False
0
```

`TryParse` writes the default value of the type into the `out` variable when the text cannot be converted, and reports the outcome with a boolean.

Example 5: the operators in one program.

C#

```
int x = 10;
x += 5;
x++;
Console.WriteLine(x);                    // 16
Console.WriteLine(x > 15 && x < 20);     // True
Console.WriteLine(5 & 3);                // 1
Console.WriteLine(5 | 3);                // 7
Console.WriteLine(5 ^ 3);                // 6
Console.WriteLine(1 << 4);               // 16
Console.WriteLine(32 >> 2);              // 8
Console.WriteLine(~0);                   // -1
```

output

```
16
True
1
7
6
16
8
-1
```

The bitwise operators work on the two-complement representation of the integer, which is why flipping every bit of zero gives minus one.

Example 6: the conditional operators.

C#

```
int score = 72;
string grade = score >= 90 ? "A" : score >= 80 ? "B" : "C";
Console.WriteLine(grade);                // C

string? maybe = null;
Console.WriteLine(maybe ?? "none");      // none
Console.WriteLine(maybe?.Length is null ? "(null)" : "has text");
maybe ??= "fallback";
Console.WriteLine(maybe);                // fallback
```

output

```
C
none
(null)
fallback
```

`??` picks the right side when the left side is null, `?.` returns null instead of throwing, and `??=` assigns only when the variable is still null.

Example 7: formatting numbers with an explicit culture.

C#

```
using System.Globalization;

double ratio = 0.256;
Console.WriteLine(ratio.ToString("F2", CultureInfo.InvariantCulture));   // 0.26
Console.WriteLine(12345.6.ToString("N2", CultureInfo.InvariantCulture)); // 12,345.60
Console.WriteLine(255.ToString("X"));        // FF
Console.WriteLine(42.ToString("D5"));        // 00042
Console.WriteLine(0.5.ToString("F1", CultureInfo.InvariantCulture));     // 0.5
```

output

```
0.26
12,345.60
FF
00042
0.5
```

The most used specifiers are `F` with a number of decimals for fixed point, `N` for grouped thousands, `X` for hexadecimal and `D` for zero-padded integers. On a machine whose culture is Spanish or German the same program prints a comma as the decimal separator unless the invariant provider is supplied.

2.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Dividing two integers and expecting a fraction	the fraction is discarded before you see the result	cast one operand to `double`, or write the literal as `2.0`
Writing `1.5` where an `int` is expected	no implicit conversion from a double literal to `int` exists, so the build fails	use `1.5` in a `double` variable, or write `1`
Using `float` for money	binary rounding shows up as cents of error	use `decimal` with the `m` suffix
Assuming `int` never overflows	a counter can wrap to a negative value and corrupt totals	use `checked`, a wider type, or validate the input
Using `int.Parse` on user input	an exception escapes for an ordinary typing error	use `int.TryParse` and branch on the result
Comparing strings with `<`	strings do not support the relational operators	use `string.Compare` or the `IComparable<string>` interface
Mixing `and` and `or` without parentheses	a reader must remember the precedence to interpret the condition	add parentheses even when the language does not need them
Comparing a `double` with `==`	rounding makes exact equality unreliable	compare the difference with a small tolerance

2.5 Practice exercises

1. Predict the output of `Console.WriteLine(9 / 2); Console.WriteLine(9 / 2.0);`.

2. Rewrite `if (list != null && list.Count > 0)` using a null-conditional operator.
3. Which type would you use for a bank balance, and which for a scientific measurement? Give the reason.
4. What is printed by `Console.WriteLine(3 + 4 * 2 - 1);`?
5. Write a fragment that reads a line from the console and converts it to an integer, printing `0` when the input is not a number.

Solutions

1. `4` and `4.5`. The first division is integer division, the second is promoted to a `double` because of the literal `2.0`.
2. `if (list?.Count > 0)`. The null-conditional operator yields `null` when `list` is null, and the comparison with `null` is false, so no exception is thrown.
3. A balance uses `decimal`, which stores base-ten fractions exactly and does not drift as money is added and subtracted. A measurement uses `double`, which is faster and covers a huge range with about 15 significant digits.
4. `10`, because multiplication happens first: `3 + 8 - 1`.

C#

```
string? line = Console.ReadLine();
int value = int.TryParse(line, out int parsed) ? parsed : 0;
Console.WriteLine(value);
```

2.6 Mini test

1. Which of these is a reference type? A) `int` B) `bool` C) `string` D) `char`
2. What do `7 / 2` and `7 / 2.0` produce? A) `3` and `3.5` B) `3.5` and `3.5` C) `3` and `3` D) `3.5` and `3`
3. What is the type of the variable in `var total = 3;`? A) `double` B) `int` C) `object` D) `var`
4. Which expression converts text to a number without throwing on bad input? A) `int.Parse(text)` B) `Convert.ToInt32(text)` C) `int.TryParse(text, out int n)` D) `(int)text`
5. True or false: `decimal` is the recommended type for money.
6. What does `checked(int.MaxValue + 1)` do? A) returns `int.MinValue` B) throws an `OverflowException` C) returns `0` D) does not compile
7. Which operator assigns a value only when the variable is null? A) `??` B) `??=` C) `?.` D) `?=`
8. What does `5 & 3` evaluate to? A) `1` B) `7` C) `6` D) `8`

Answer key and explanations

1. C - strings, classes and arrays are reference types, while the numeric types and `bool` are value types.
2. A - integer division truncates to `3`, and the presence of a `double` literal promotes the operation to floating point.
3. B - the literal `3` is an `int`, and `var` only asks the compiler to infer that type.
4. C - `TryParse` reports failure through its return value instead of an exception.

5. True - `decimal` keeps base-ten fractions exact, which is what currency requires.
6. B - `checked` turns silent wrapping into an `OverflowException`.
7. B - the compound coalescing assignment.
8. A - `5` is `101` and `3` is `011` in binary, so the bitwise `and` leaves only the lowest bit set.

Chapter 3 - Control flow: decisions and loops

3.1 Summary

- A branch runs a block only when its condition is true, so the order of the tests decides the outcome.
- `if`, `else if` and `else` handle a chain of tests, and `switch` handles one value with many arms.
- A condition must be a `bool`; unlike JavaScript or Python, C# has no truthy values.
- The ternary operator picks one of two values, which makes it useful in initialisers.
- A `switch` statement compares constants and patterns, and an arm ends with `break` or another jump statement.
- `while` tests before the body, `do` tests after it, `for` bundles the counter, and `foreach` walks a sequence.
- `break` leaves the nearest loop or switch, `continue` jumps to the next pass, and `return` leaves the method.
- A variable declared inside a block lives only until the closing brace, which is its scope.
- Nested loops multiply their passes, so the body of a 10 by 10 grid runs one hundred times.
- Every loop needs a way to end, so the condition or the counter must change inside the body.

3.2 Concepts

How `if` decides

output

```
             +-------------------+
condition -| true              | false
             v                   v
       +-----+------+      +-----+------+
       | then block |      | else block |
       +------------+      +------------+
```

The condition is any expression of type `bool`. Braces are optional for a single statement, but this volume always writes them, because adding a second statement to a braceless `if` is a classic source of bugs.

C#

```
int temperature = 24;
if (temperature > 30)
{
    Console.WriteLine("hot");
}
else if (temperature > 15)
{
    Console.WriteLine("warm");
}
else
{
    Console.WriteLine("cold");
}
```

output

```
warm
```

The tests run in order and only the first matching block executes, so place the most specific test first. A `> 0` test placed before a `> 100` test makes the second one unreachable.

Boolean building blocks

Operator	Meaning		
`!`	negation, as in `!ready`		
`&&`	logical and, short-circuits		
`|`		`|`	logical or, short-circuits
`&`, `|`	`|`	the same without short-circuiting, rarely needed	
`==`, `!=`	equality and inequality		
`<`, `>`, `<=`, `>=`	ordering		

Short-circuiting skips the right operand when the left one decides the result, which is what makes the guard `list is not null && list.Count > 0` safe. Prefer `is null` and `is not null` over `== null`, because those patterns never call a user-defined equality operator.

The ternary operator

The conditional operator `condition ? valueWhenTrue : valueWhenFalse` returns one of two values. Both branches must have compatible types, and the operator is an expression rather than a statement, so it may appear anywhere a value is expected.

C#

```
int score = 72;
string grade = score >= 90 ? "A" : score >= 80 ? "B" : "C";
Console.WriteLine(grade);                          // C
Console.WriteLine(score % 2 == 0 ? "even" : "odd");  // even
```

The `switch` statement

A `switch` statement evaluates one expression and runs the block of the first matching arm. Unlike C and C++, an arm that contains statements ends with `break` or with another jump statement; falling into the next arm is allowed only when a case has an empty body, which is how several constants share one implementation.

C#

```
string day = "Saturday";
switch (day)
{
    case "Monday":
    case "Tuesday":
    case "Wednesday":
    case "Thursday":
    case "Friday":
        Console.WriteLine("weekday");
        break;
    case "Saturday":
    case "Sunday":
        Console.WriteLine("weekend");
        break;
    default:
        Console.WriteLine("not a day");
        break;
}
```

output

```
weekend
```

Since C# 7 a case label may also be a pattern, which is how types, ranges and property tests appear in a switch. Chapter 9 covers pattern matching in depth, including the `switch` expression that returns a value instead of running statements.

The `while` and `do` loops

`while (condition) { body }` tests before each pass, so the body may never run. `do { body } while (condition);` tests after the first pass, so the body always runs at least once, which suits menus and prompts.

C#

```
int countdown = 3;
while (countdown > 0)
{
    Console.WriteLine(countdown);
    countdown--;
}
```

output

```
3
2
1
```

The `for` loop

A `for` loop brings the three parts of a counter loop into one line: the initialiser runs once, the condition is tested before each pass, and the iterator runs after each pass.

C#

```
for (int i = 0; i < 5; i++)
{
    Console.WriteLine($"square of {i} is {i * i}");
}
```

output

```
square of 0 is 0
square of 1 is 1
square of 2 is 4
square of 3 is 9
square of 4 is 16
```

A `for` loop may declare several counters and update several values, and any of the three parts may be empty, although an empty condition means an endless loop until a `break`.

The `foreach` loop

`foreach` walks anything enumerable: an array, a list, a string, a dictionary or any type that exposes an enumerator. It has no index, so it is the safe choice when only the sequence matters, and it reads faster than a counted loop.

C#

```
string[] languages = ["C#", "F#", "Visual Basic"];
foreach (string language in languages)
{
    Console.WriteLine(language.ToUpperInvariant());
}

foreach (char letter in "hi")
{
    Console.WriteLine(letter);
}
```

output

```
C#
F#
VISUAL BASIC
h
i
```

Modifying a collection while a `foreach` walks it is forbidden for `List<T>` and throws an `InvalidOperationException`. Iterate over a copy such as `list.ToArray()` when the loop removes elements.

Jump statements and scope

Statement	Effect
`break`	leaves the nearest loop or `switch`

Statement	Effect
`continue`	skips the rest of this pass and starts the next one
`return value`	leaves the method, optionally with a result
`goto case "x"`	jumps to another arm of the same switch, rarely used

A block is the region between braces. Variables declared in it are created when the block is entered and discarded when it is left, so they are invisible after the closing brace and cannot be redeclared in a nested block. That is why a `for` counter is declared in the header: the compiler sees the loop as its owner.

C#

```
for (int i = 0; i < 3; i++)
{
    int doubled = i * 2;
    Console.WriteLine(doubled);
}
// i and doubled are not visible here
```

3.3 Examples

Example 1: an if chain that classifies a number.

C#

```
int value = -4;
if (value < 0)
{
    Console.WriteLine("negative");
}
else if (value == 0)
{
    Console.WriteLine("zero");
}
else if (value < 10)
{
    Console.WriteLine("small positive");
}
else
{
    Console.WriteLine("large positive");
}
```

output

```
negative
```

Example 2: a switch over an enumeration.

C#

```
enum Season { Spring, Summer, Autumn, Winter }

Season current = Season.Summer;
switch (current)
{
    case Season.Spring:
        Console.WriteLine("plant");
        break;
    case Season.Summer:
        Console.WriteLine("harvest");
        break;
    case Season.Autumn:
        Console.WriteLine("gather");
        break;
    case Season.Winter:
        Console.WriteLine("rest");
        break;
}
```

output

```
harvest
```

An `enum` declares named constants of an integral type. `(int)Season.Summer` is `1`, because the first member of an enumeration is zero unless a value is given.

Example 3: a do loop that keeps asking until the input is a number.

C#

```
int number = 0;
bool valid = false;
do
{
    Console.Write("enter a number: ");
    string? line = Console.ReadLine();
    valid = int.TryParse(line, out number);
}
while (!valid);

Console.WriteLine($"you entered {number}");
```

output

```
enter a number: 12
you entered 12
```

The output assumes the user typed `12`. The loop repeats while the conversion fails, which is the shape of every prompt in a console program.

Example 4: a while loop that accumulates.

C#

```
int total = 0;
int next = 1;
while (total < 20)
{
    total += next;
    next++;
}
Console.WriteLine($"total {total}, steps {next - 1}");
```

output

```
total 21, steps 6
```

The running sum reaches 15 after five steps and 21 after six, and 21 is the first total that is not below 20.

Example 5: nested loops that build a multiplication grid.

C#

```
for (int row = 1; row <= 3; row++)
{
    string line = "";
    for (int column = 1; column <= 3; column++)
    {
        line += $"{row * column,4}";
    }
    Console.WriteLine(line);
}
```

output

```
   1   2   3
   2   4   6
   3   6   9
```

The inner loop runs three times for each of the three outer passes, so the body executes nine times. The `4` inside the interpolation pads each number to a width of four characters, which keeps the columns aligned.

Example 6: break and continue.

C#

```
for (int i = 1; i <= 10; i++)
{
    if (i == 6)
    {
        break;
    }
    if (i % 2 == 0)
    {
        continue;
    }
    Console.WriteLine(i);
}
```

output

```
1
3
5
```

The even values are skipped and the loop stops when the counter reaches six.

Example 7: foreach over a list with a filter.

C#

```
List<int> values = [4, 7, 10, 13, 16];
foreach (int value in values)
{
    if (value % 2 == 1)
    {
        Console.WriteLine($"{value} is odd");
    }
}
```

output

```
7 is odd
13 is odd
```

3.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Writing `=` instead of `==` in a condition	the build rejects the assignment, although the intent was a comparison	write `==` inside the condition
Writing a non-boolean condition such as `if (count)`	C# requires a `bool`, so this does not compile	use `if (count > 0)`
Forgetting to change the loop variable in a `while` loop	the condition never becomes false and the program hangs	update the variable in the body, or use a `for` loop
Modifying a list while `foreach` walks it	throws `InvalidOperationException` at the moment of the change	iterate over `list.ToArray()`, or count backwards with an index
Testing the general case before the specific one	the specific arm becomes unreachable and silent	order the tests from specific to general
Omitting braces around a single statement body	adding a second statement silently detaches it from the condition	always use braces and let the editor format them
Using `else` after a block that always returns	the code after the `else` can never run	return early and drop the `else`
Using the row index as a stable key	the index changes when the list is sorted	use the value itself, or an id field

3.5 Practice exercises

1. Write an `if` chain that prints `child`, `teen` or `adult` for ages below 13, below 18, and 18 or above.
2. What is printed by a `do` loop whose condition is false from the start?
3. Why does `for (int i = 0; i < list.Count; i++)` misbehave when the body removes elements from the list?
4. Rewrite the `while` loop that sums the numbers from 1 to 10 as a `for` loop.
5. Predict the output of nested loops that print `$"{row}{column}"` for rows 1 to 2 and columns 1 to 2.

Solutions

C#

```
if (age < 13) Console.WriteLine("child");
else if (age < 18) Console.WriteLine("teen");
else Console.WriteLine("adult");
```

2. The body still runs once, because `do` tests the condition after the body instead of before it.
3. Removing an element shifts the following elements down, so the loop skips one item, and the count changes while the loop is running. Iterate backwards from `list.Count - 1`, or iterate over a copy.

C#

```
int total = 0;
for (int i = 1; i <= 10; i++)
{
    total += i;
}
```

5. `11`, `12`, `21` and `22`, each on its own line, because the inner loop runs twice for each of the two outer passes.

3.6 Mini test

1. Which condition is legal in C#? A) `if (count)` B) `if (count > 0)` C) `if (count = 0)` D) `if (count) then`
2. What does `&&` do that `&` does not? A) it works on booleans B) it skips the right operand when the left one decides the result C) it returns an integer D) it works on strings
3. True or false: the body of a `do` loop always runs at least once.
4. Which statement skips the rest of a loop pass? A) `break` B) `continue` C) `return` D) `goto`
5. What happens when a `foreach` body adds an item to the list being walked? A) the new item is visited B) an `InvalidOperationException` is thrown C) the loop ends normally D) the compiler rejects the code
6. How long does a variable declared inside a loop body live? A) for the whole program B) until the method returns C) only until the closing brace of that block D) it is stored on the

heap

7. What does `is not null` test? A) whether the value is an instance of a type B) whether the reference is not the null value C) whether the type is not nullable D) whether the value is zero
8. Which header describes a counted loop? A) `while (i < 10)` B) `do { } while (i < 10);` C) `for (int i = 0; i < 10; i++)` D) `foreach (int i in range)`

Answer key and explanations

1. B - a condition must be a boolean expression, and `=` is assignment.
2. B - short-circuiting is what makes a null guard safe.
3. True - the test happens after the first pass, so a false condition cannot prevent it.
4. B - `continue` moves to the next iteration, while `break` leaves the loop entirely.
5. B - the enumerator detects the change and throws, which protects the iteration from shifting indices.
6. C - a block owns the variables declared in it, which is the definition of scope.
7. B - the `is` patterns compare against the null value directly.
8. C - the `for` header holds the initialiser, the condition and the iterator.